

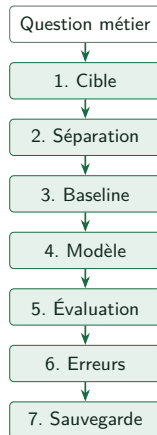
ML supervisé de bout en bout

De la question métier au modèle décisionnel

M. ATARMLA Abdou-Raouf Pr. PINDRA Nadjime

Togo AI Summer School 2026 · Filière F1

Mardi 25 août 2026



Le principe de la séance

Chaque notion mathématique est suivie, dans les vingt minutes, de la ligne de code qui l'applique.

À retenir dès maintenant

Le score d'un modèle n'est pas un nombre. C'est un protocole.

Situation

Une ONG suit 2 400 exploitations agricoles dans les cinq régions du Togo. Chaque campagne, une partie subit des pertes post-récolte supérieures à 20 %. Le budget d'accompagnement est limité : formation au stockage, accès à un magasin communautaire, appui à la commercialisation.

À qui adresser cet accompagnement, avant la récolte ?

Cible binaire

la décision l'est aussi

Classe rare : 14 %

l'accuracy va mentir

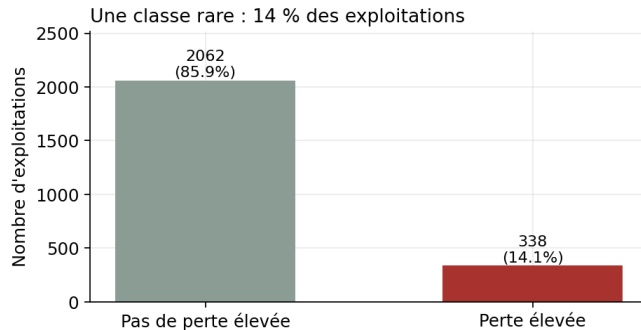
Coût asymétrique

le seuil devient un choix

Colonne	Type	Remarque
region, culture_principale	catégorielle	5 et 5 modalités
type_sol, type_stockage	catégorielle	valeurs manquantes sur le sol
superficie_ha	numérique	hectares
distance_marche_km	numérique	valeurs manquantes
pluviometrie_mm	numérique	valeurs manquantes
age_exploitant	numérique	
delai_avant_vente_j	numérique	valeurs manquantes
irrigation, formation_recue	binaire	
reclamation_appui_urgence	binaire	nous y reviendrons
perte_elevee	cible	14,1 % de positifs

2 400 lignes, 13 variables explicatives, une cible.

Une classe rare



338 exploitations à risque sur 2 400

M1 · Généralisation et compromis biais-variance

Pr. PINDRA Nadjime · 10 minutes

Que veut dire apprendre ?

On dispose de n observations (x_i, y_i) tirées indépendamment d'une loi $\mathcal{D}$ **inconnue**.
On cherche une fonction f qui prédit y à partir de x .

Ce que l'on mesure

$$\hat{R}_n(f) = \frac{1}{n} \sum_{i=1}^n \ell(y_i, f(x_i))$$

L'erreur **empirique**, sur les données dont on dispose.

Ce qui nous intéresse

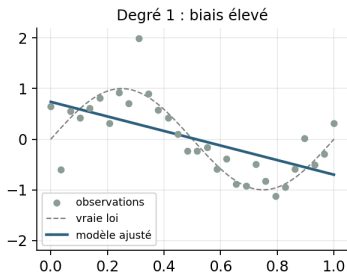
$$R(f) = \mathbb{E}_{(x,y) \sim \mathcal{D}} [\ell(y, f(x))]$$

L'erreur de **généralisation**, sur toute la population.

Si f est fixée *avant* de voir les données, $\hat{R}_n(f)$ est un estimateur sans biais de $R(f)$.

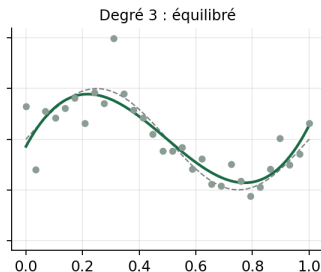
Mais f est choisie en minimisant $\hat{R}_n$. L'estimateur devient optimiste.

Trop simple, trop riche



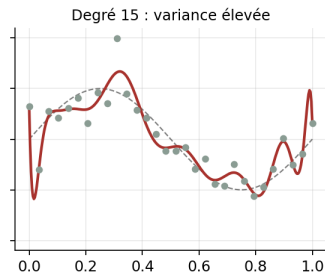
Biais élevé

le modèle ne peut pas représenter la structure



Équilibre

la structure est captée, le bruit ignoré



Variance élevée

le modèle apprend le bruit de cet échantillon

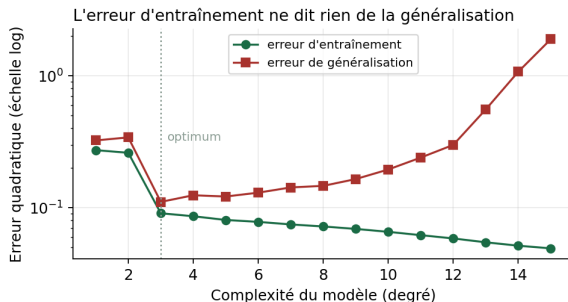
Pour un modèle de régression et une perte quadratique, en un point x :

$$\mathbb{E}[(y - \hat{f}(x))^2] = \underbrace{(\mathbb{E}[\hat{f}(x)] - f^*(x))^2}_{\text{biais}^2} + \underbrace{\mathbb{V}[\hat{f}(x)]}_{\text{variance}} + \underbrace{\sigma^2}_{\text{bruit}}$$

- Le **biais** diminue quand la classe de modèles s'enrichit.
- La **variance** augmente avec cet enrichissement.
- Le **bruit** est irréductible : aucune quantité de données ne l'élimine.

L'erreur totale passe donc par un minimum, à une complexité intermédiaire.

Ce que l'on voit sur les données



L'erreur d'entraînement décroît toujours. L'erreur de généralisation remonte.

Mesurer sur les données d'entraînement ne permet pas de choisir un modèle.

E2 · Cadrer le problème

ATARMLA · 6 minutes

La demande de l'ONG, telle qu'elle arrive :

« À qui adresser notre accompagnement ? »

Ce n'est pas encore un problème supervisé. Il le devient quand on écrit :

Prédire $\text{perte_elevee} \in \{0, 1\}$ à partir des seules informations disponibles avant la récolte.

La clause temporelle n'est pas un détail de rédaction. C'est elle qui détermine quelles colonnes ont le droit d'entrer dans le modèle.

Trois demandes qui ne sont pas supervisables

Demande	Ce qui bloque
« Segmenter nos exploitants en profils types »	Pas de cible. Problème non supervisé.
« Prédire la perte » alors qu'elle n'a jamais été mesurée	Pas de labels. Il faut d'abord collecter.
« Prédire la perte à partir du montant d'indemnisation versé »	La variable n'existe pas au moment de la décision.

Le troisième cas n'est pas une erreur de débutant. C'est le piège le plus fréquent en pratique, et le sujet de la séquence suivante.

Classification ou régression ?

Le taux de perte est une grandeur continue. Nous le seuillons à 20 % pour en faire une cible binaire.

Ce que l'on gagne

La décision de l'ONG est binaire : accompagner ou non. Le modèle parle le même langage que la décision.

Ce que l'on perd

L'information de gravité. Une perte de 21 % et une perte de 60 % deviennent indistinguables.

C'est un choix de modélisation, pas une évidence. Il doit être assumé et documenté.

E3 · Séparer, et la fuite de données

ATARMLA · 11 minutes

La séparation, avant tout le reste

```
from sklearn.model_selection import train_test_split

X = df.drop(columns=["perte_elevee", "id_exploitation"])
y = df["perte_elevee"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=0
)
```

stratify=y

Avec 14 % de positifs, un tirage libre peut produire un test à 10 % ou à 18 %. La comparaison devient impossible.

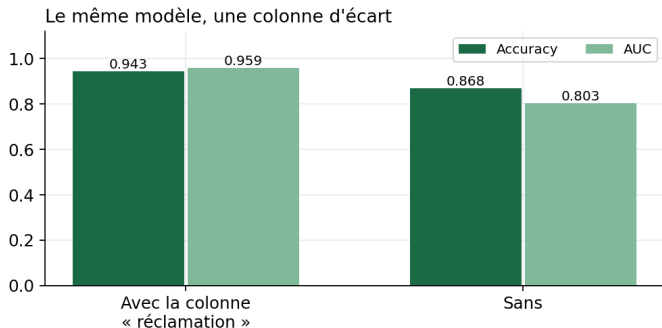
random_state=0

La reproductibilité n'est pas un confort. C'est ce qui rend un résultat discutable entre collègues.

J'ai un modèle à **94,3 %** d'accuracy
sur des données qu'il n'a jamais vues.

Qui le met en production demain ?

Le même modèle, une colonne d'écart



`reclamation_appui_urgence` : l'exploitant a déposé une demande d'appui d'urgence.

La chronologie

La réclamation est déposée **après** la récolte, une fois la perte constatée. Elle est une **conséquence** de la cible, pas une cause.

En production, au moment où l'ONG doit décider qui accompagner, cette colonne est vide pour tout le monde. Le modèle à 94 % ne peut tout simplement pas fonctionner.

Une variable indisponible au moment de la prédiction est une fuite, même corrélée, même si elle améliore le score. Surtout si elle l'améliore beaucoup.

Trois autres formes de fuite, plus discrètes

1. **Normaliser avant de séparer.** La moyenne et l'écart-type utilisés sur le train contiennent de l'information venue du test.
2. **Imputer avec la statistique globale.** Même mécanisme : la médiane calculée sur l'ensemble fuit vers le train.
3. **Dédupliquer après avoir séparé.** Le même exploitant se retrouve des deux côtés, et le modèle le reconnaît au lieu de généraliser.

Le point important

Ces trois fuites ne se voient pas dans le score. Elles le rendent seulement trop beau. Nous allons les rendre **mécaniquement impossibles** avec un Pipeline.

E4 · La baseline

ATARMLA · 5 minutes

```
from sklearn.dummy import DummyClassifier  
  
dummy = DummyClassifier(strategy="most_frequent").fit(X_train, y_train)  
print(f"Baseline : {dummy.score(X_test, y_test):.3f}")
```

0,860

Ce classifieur répond « pas de perte élevée » à tout le monde.
Il n'a rien appris. Il n'a même pas regardé les données.

86 % de bonnes réponses.

Zéro exploitation accompagnée.

Un modèle peut être excellent sur la métrique et totalement inutile pour la décision.

Un modèle sans baseline n'a pas de score.

M2 · La régression logistique

Pr. PINDRA Nadjime · 12 minutes

Pourquoi pas une régression linéaire ?

La cible vaut 0 ou 1. On pourrait ajuster $\hat{y} = \beta_0 + \beta^\top x$ par moindres carrés. Deux objections.

Le domaine

$\beta_0 + \beta^\top x$ parcourt $\mathbb{R}$ tout entier. Une prédiction de $-0,3$ ou de $1,4$ n'a aucun sens pour une probabilité.

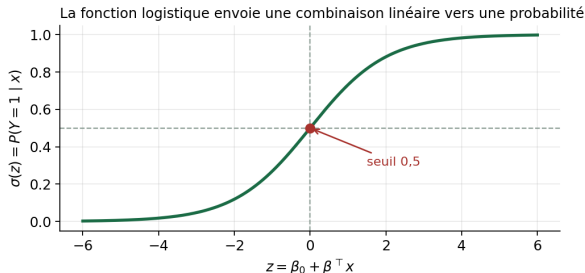
La variance

Pour une variable de Bernoulli, $\mathbb{V}[y \mid x] = p(x)(1 - p(x))$ dépend de x . L'hypothèse d'homoscédasticité des moindres carrés est violée.

Il faut une fonction qui envoie $\mathbb{R}$ dans $[0, 1]$.

La fonction logistique

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad z = \beta_0 + \beta^\top x$$



$$P(Y = 1 \mid x) = \sigma(\beta_0 + \beta^\top x) \quad \Longleftrightarrow \quad \log \frac{p}{1-p} = \beta_0 + \beta^\top x$$

Sous un modèle de Bernoulli, la vraisemblance de l'échantillon s'écrit

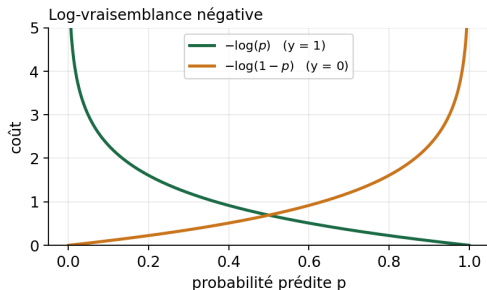
$$L(\beta) = \prod_{i=1}^n p_i^{y_i} (1 - p_i)^{1-y_i}, \quad p_i = \sigma(\beta_0 + \beta^\top x_i)$$

En passant au logarithme et en changeant de signe, on minimise

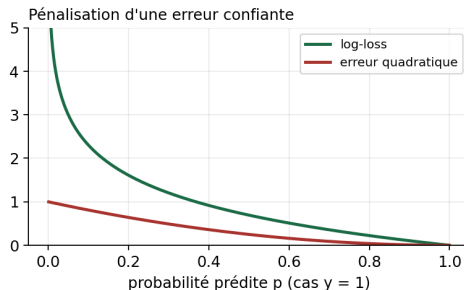
$$\mathcal{L}(\beta) = - \sum_{i=1}^n \left[y_i \log p_i + (1 - y_i) \log(1 - p_i) \right]$$

C'est l'estimateur du maximum de vraisemblance. La « log-loss » n'est pas une métrique arbitraire : c'est la vraisemblance du modèle.

Pourquoi pas l'erreur quadratique ?



Une prédiction confiante et fausse ($p \rightarrow 0$ alors que $y = 1$) est pénalisée **infiniment** par la log-loss.



L'erreur quadratique la borne à 1. Elle est de plus **non convexe** en β après composition par σ .

Le gradient prend une forme remarquablement simple :

$$\nabla_{\beta} \mathcal{L} = \sum_{i=1}^n (p_i - y_i) x_i = X^{\top} (p - y)$$

et la matrice hessienne s'écrit

$$H = X^{\top} W X, \quad W = \text{diag}(p_i(1 - p_i)) \succeq 0$$

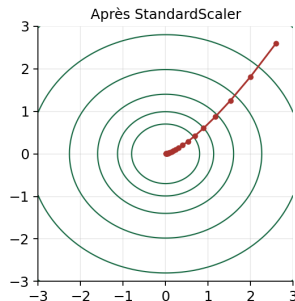
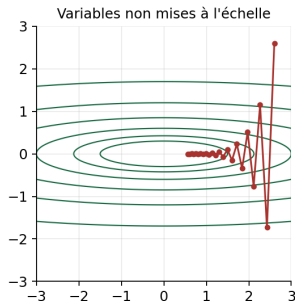
Conséquence

H est semi-définie positive, donc $\mathcal{L}$ est convexe. Il n'y a **pas de minimum local parasite**. Tout algorithme de descente convergera vers la même solution, quel que soit le point de départ.

Il n'existe pas de solution close : l'optimisation est itérative. C'est ce que gouverne l'argument `max_iter` que nous verrons dans le code. La descente de gradient elle-même est le sujet de la plénière de demain matin.

Pourquoi mettre les variables à l'échelle

Les valeurs propres de $H = X^T W X$ dépendent des unités des colonnes. Une variable en kilomètres (jusqu'à 100) et une en hectares (jusqu'à 10) produisent un rapport de valeurs propres élevé : le problème est **mal conditionné**.



Le StandardScaler du pipeline n'est pas une coquetterie. Il conditionne la convergence.

$$\log \frac{p}{1-p} = \beta_0 + \beta_1 x_1 + \cdots + \beta_k x_k$$

Augmenter x_j d'une unité multiplie la cote $\frac{p}{1-p}$ par e^{β_j} .

Exemple de lecture

Un coefficient $\beta_j = 0,7$ pour « stockage : aucun » signifie que l'absence de stockage multiplie la cote de perte élevée par $e^{0,7} \approx 2$, toutes choses égales par ailleurs.

C'est ce qui rend ce modèle défendable devant l'ONG : chaque coefficient s'explique en une phrase.

E5 · Construire le pipeline

ATARMLA · 10 minutes

Le prétraitement, par type de colonne

```
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.pipeline import Pipeline

num = X_train.select_dtypes(include="number").columns.tolist()
cat = X_train.select_dtypes(exclude="number").columns.tolist()

prep = ColumnTransformer([
    ("num", Pipeline([("imp", SimpleImputer(strategy="median")),
                      ("sc", StandardScaler())])), num),
    ("cat", Pipeline([("imp", SimpleImputer(strategy="most_frequent")),
                      ("oh", OneHotEncoder(handle_unknown="ignore"))])), cat),
])
```

`handle_unknown="ignore"` : une modalité présente en test mais absente du train ne fait pas échouer la prédiction.

Le modèle complet en un objet

```
from sklearn.linear_model import LogisticRegression

modele = Pipeline([("prep", prep),
                   ("clf", LogisticRegression(max_iter=1000))])

modele.fit(X_train, y_train)
print(f"Score : {modele.score(X_test, y_test):.3f}")
```

StandardScaler

le conditionnement démontré à l'instant

max_iter=1000

le budget de l'optimisation itérative

Imputation dans le pipeline

recalculée sur le train seul, à chaque pli

Les trois fuites discrètes de la séquence E3 sont désormais impossibles.

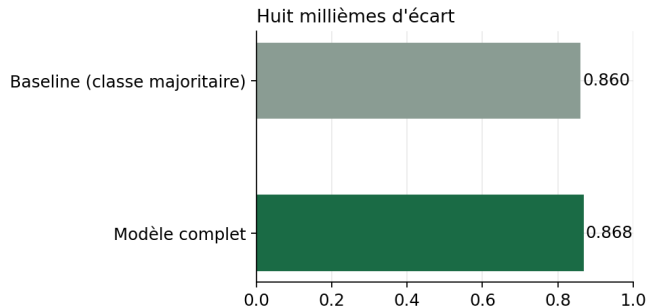
Le modèle ne prédit pas une classe

```
modele.predict_proba(X_test)[:3]
# array([[0.94, 0.06],
#        [0.71, 0.29],
#        [0.88, 0.12]])
```

Il produit la probabilité $\sigma(\beta_0 + \beta^\top x)$ que le Pr. PINDRA vient d'introduire.

Ce que fait predict

predict applique simplement le seuil 0,5 à cette probabilité. Nous verrons que ce 0,5 est une convention, et non un résultat.



Nous avons gagné **huit millièmes** sur un modèle qui ne fait rien.

Le modèle est-il mauvais, ou la métrique est-elle mauvaise ?

M3 · Que vaut ce score ?

Pr. PINDRA Nadjime · 20 minutes

L'accuracy est une moyenne pondérée

Notons $\pi = P(Y = 1)$ la prévalence. Pour un classifieur quelconque,

$$\text{accuracy} = \pi \cdot \underbrace{P(\hat{Y} = 1 \mid Y = 1)}_{\text{rappel}} + (1 - \pi) \cdot \underbrace{P(\hat{Y} = 0 \mid Y = 0)}_{\text{spécificité}}$$

Avec $\pi = 0,141$, le second terme pèse **six fois** plus que le premier.

Conséquence

Un classifieur constant à 0 obtient $1 - \pi = 0,859$ sans jamais rien détecter. Sur un phénomène rare, l'accuracy mesure surtout la rareté du phénomène.

Les quatre nombres qui comptent

	prédit 0	prédit 1
réel 0	vrais négatifs	faux positifs
réel 1	faux négatifs	vrais positifs

$$\text{précision} = \frac{VP}{VP + FP}$$

Quand le modèle signale, a-t-il raison ?

$$\text{rappel} = \frac{VP}{VP + FN}$$

Parmi ceux qu'il fallait trouver, combien sont trouvés ?

Notations utilisées dans le code : `y_true` et `y_pred`.

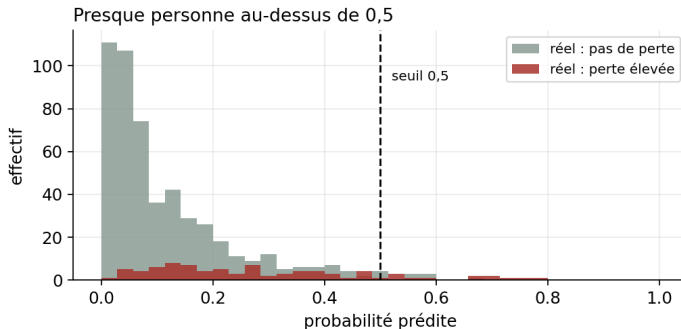
	prédit 0	prédit 1
réel 0	508	8
réel 1	71	13

$$\text{précision} = \frac{13}{21} = 0,62$$

$$\text{rappel} = \frac{13}{84} = \mathbf{0,155}$$

86,8 % d'accuracy, et 71 exploitations à risque sur 84 laissées de côté.

Pourquoi si peu de détections



Le modèle sépare correctement les deux populations, mais presque aucune exploitation ne dépasse la probabilité 0,5.

Le seuil est une décision, pas un résultat

Notons C_{FN} le coût d'une exploitation à risque manquée et C_{FP} celui d'une visite inutile. Pour une exploitation de probabilité prédite p , le coût espéré est

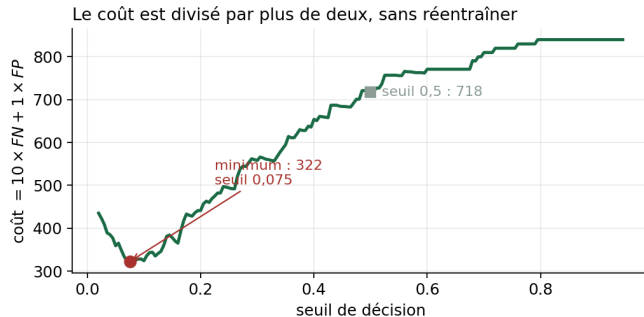
$$\text{signaler : } (1 - p) C_{FP} \quad \text{contre} \quad \text{ne pas signaler : } p C_{FN}$$

On a donc intérêt à signaler dès que

$$p C_{FN} > (1 - p) C_{FP} \quad \Longleftrightarrow \quad p > \frac{C_{FP}}{C_{FP} + C_{FN}}$$

Application

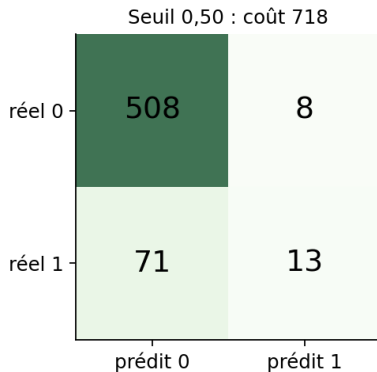
Si manquer une exploitation coûte dix fois une visite inutile : $t^* = \frac{1}{1+10} \approx 0,09$, et non 0,5.



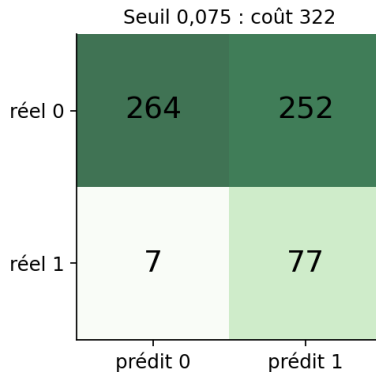
Coût = $10 \times FN + 1 \times FP$. Seuil 0,5 : **718**. Seuil 0,075 : **322**.

Le coût est divisé par plus de deux, sans réentraîner et sans donnée supplémentaire.

Les deux matrices

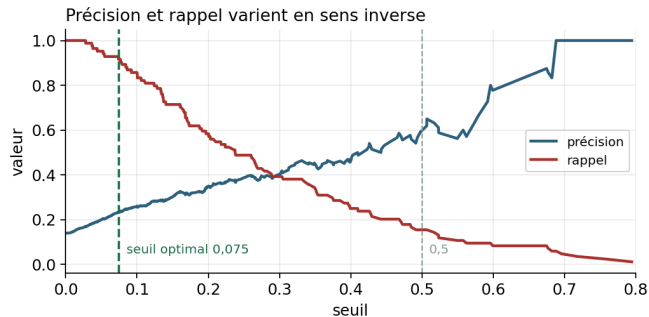


Seuil 0,5 : 13 détections, 71 manquées.
Rappel 0,155



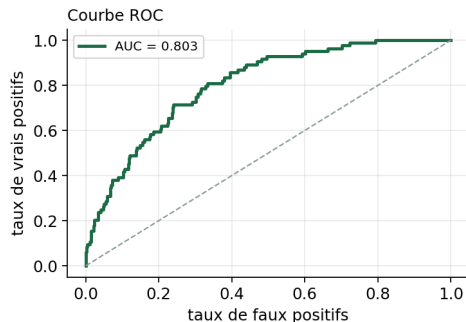
Seuil 0,075 : 77 détections, 7 manquées.
Rappel 0,917, au prix de 252 visites inutiles

Le compromis, en continu

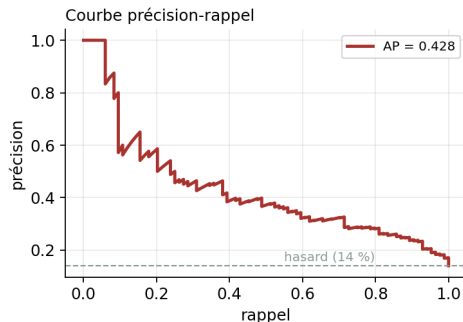


Précision et rappel varient en sens inverse. Aucun seuil ne les maximise ensemble.

Deux courbes de synthèse

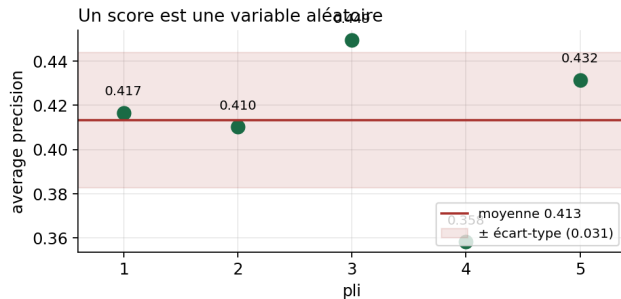


La **ROC** est indépendante de la prévalence.
 $AUC = 0,803$.



La courbe **précision-rappel** est plus honnête
sur données rares : le hasard s'y situe à 14 %,
pas à 50 %.

Un score est une variable aléatoire



Le score dépend du découpage. Sur cinq plis, l'écart-type vaut 0,031.

Un écart de 0,02 entre deux modèles n'est pas une différence.

E6 · Atelier

Trinômes · 24 minutes

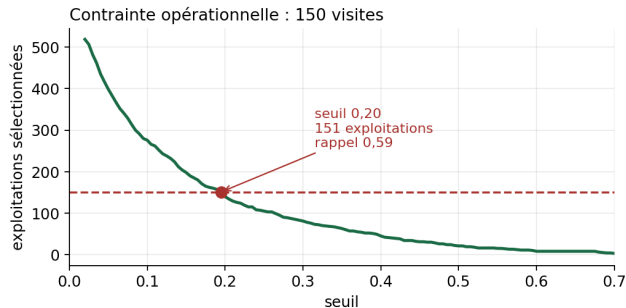
Contrainte opérationnelle

L'ONG peut accompagner **150 exploitations** cette campagne. Votre modèle doit lui dire lesquelles. **Justifiez votre seuil.**

Tâche	Durée
1. Exécuter le pipeline fourni, récupérer les probabilités sur le test	4 min
2. Matrice de confusion au seuil 0,5. Combien d'exploitations manquées ?	4 min
3. Faire varier le seuil de 0,05 à 0,60, tracer précision et rappel	7 min
4. Choisir le seuil qui remplit exactement le budget de 150 visites	5 min
5. Rédiger en deux phrases la recommandation à l'ONG	4 min

Débloccage : trinôme, puis groupe voisin, puis énoncé, puis main levée.

La réponse attendue



Seuil 0,195 : 151 exploitations sélectionnées, rappel de 0,595.
L'ONG détecte 50 exploitations à risque sur 84 avec son budget réel.

Comment votre travail sera regardé

Critère	Insuffisant	Attendu
Choix du seuil	« 0,5 par défaut »	Seuil relié au budget de 150 visites
Lecture de la matrice	Cite l'accuracy	Cite le rappel et le nombre d'exploitations manquées
Recommandation	Décrit le modèle	S'adresse à l'ONG en termes de décision
Honnêteté	Annonce le meilleur chiffre	Annonce ce que le modèle ne sait pas faire

Le groupe qui nous dira ce que son modèle ne sait pas faire aura mieux travaillé que celui qui aura le plus gros chiffre.

E7 · Erreurs, sauvegarde, clôture

ATARMLA · 6 minutes

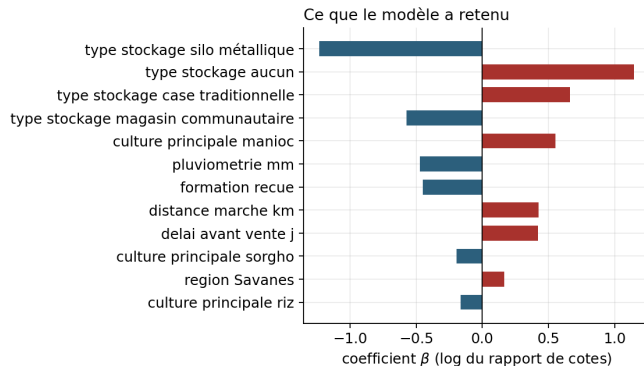
Extraire les dix faux négatifs les plus confiants et les lire un par un.

Ce que l'on cherche

Un motif : ces exploitations partagent-elles un type de stockage, une distance au marché, un délai de vente ? Si oui, il manque une variable au modèle, ou une interaction.

Dix minutes passées à regarder des erreurs valent mieux que dix modèles de plus.

Ce que le modèle a retenu



À confronter au terrain. Un coefficient qui contredit l'expérience des agents est un signal, pas une découverte.

```
import joblib

joblib.dump({"modele": modele, "seuil": 0.195},
            "modele_pertes.joblib")
```

Le seuil fait partie du modèle

Un pipeline sauvegardé sans son seuil est incomplet. C'est l'erreur classique au moment de la mise en production : le modèle est déployé, et `predict` applique silencieusement 0,5.

1. Un score se vérifie avant de se célébrer.
2. Un score n'existe que relativement à une baseline.
3. Le modèle produit une probabilité. C'est l'humain qui décide.

- **Cet après-midi**, Bloc B : le même pipeline sur d'autres données agricoles.
- **Mercredi** : c'est ce fichier `.joblib` qui sera exposé en API.
- **Jeudi** : c'est ce score que l'on surveillera en production.
- **Vendredi** : le protocole d'évaluation sera le critère de la restitution.

Prenez dix secondes.

Pensez à un jeu de données de votre travail.

Quelle colonne serait une fuite ?

Compléments

Ce que la séance ne peut pas traiter en deux heures,
et que le support conserve comme référence

Où se situe ce que nous avons fait

Famille	Ce dont on dispose	Exemples
Supervisé	des couples (x, y) étiquetés	classification, régression, prévision
Non supervisé	des x seulement	segmentation d'exploitants, réduction de dimension, détection d'anomalies
Semi-supervisé	peu d'étiquettes, beaucoup de x	annotation coûteuse, imagerie satellite
Renforcement	un signal de récompense différé	pilotage, allocation séquentielle

La question « ai-je des étiquettes ? » précède toujours le choix de l'algorithme.

- **Instance** : une ligne, ici une exploitation
- **Variable explicative** : une colonne d'entrée, x_j
- **Cible** : la colonne à prédire, y
- **Étiquette** : la valeur observée de y
- **Entraînement** : estimation de β
- **Inférence** : application à de nouvelles données
- **Hyperparamètre** : ce qui n'est pas appris, choisi par validation
- **Paramètre** : ce que l'optimisation détermine
- **Généralisation** : performance hors échantillon

Distinction souvent confondue

Le **seuil** de décision n'est ni un paramètre ni un hyperparamètre du modèle. C'est une décision de gestion, prise après l'entraînement.

Tâche	Cible	Métriques usuelles
Classification binaire	$\{0, 1\}$	accuracy, précision, rappel, F1, AUC, average precision
Classification multi-classe	$\{1, \dots, K\}$	accuracy, F1 macro et micro, matrice $K \times K$
Régression	$\mathbb{R}$	RMSE, MAE, R^2 , erreur relative
Ordinal	ordre sans échelle	kappa pondéré
Comptage	$\mathbb{N}$	déviance de Poisson

Le F1 est la moyenne harmonique de la précision et du rappel : $F_1 = 2 \frac{\text{prec} \cdot \text{rappel}}{\text{prec} + \text{rappel}}$. Il suppose implicitement que les deux comptent autant, ce qui est rarement vrai en gestion.

Catégorielles

- **One-hot** : une colonne par modalité, sans ordre implicite
- **Ordinal** : uniquement si un ordre existe réellement
- **Par la cible** : puissant et dangereux, fuite garantie hors pipeline

Numériques

- **Standardisation** : $(x - \mu) / \sigma$
- **Min-max** : ramène à $[0, 1]$
- **Logarithme** : pour les distributions à longue traîne, comme la superficie
- **Discrétisation** : perd de l'information, gagne en robustesse

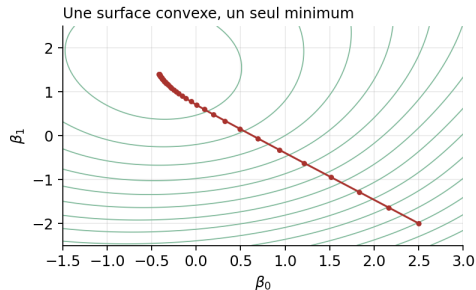
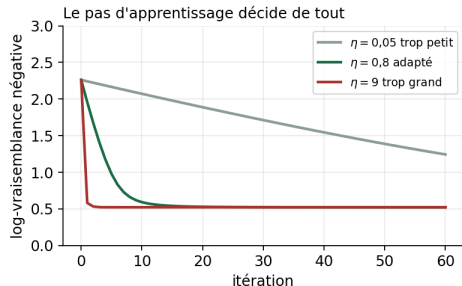
Valeurs manquantes

Imputer par la médiane est une valeur par défaut, pas une réponse. Une valeur manquante peut être **informative** : ajouter un indicateur binaire de manquant préserve cette information.

Comment l'optimisation se déroule réellement

Pour la régression logistique, la mise à jour s'écrit

$$\beta^{(t+1)} = \beta^{(t)} - \eta \nabla_{\beta} \mathcal{L}(\beta^{(t)}) = \beta^{(t)} - \eta X^{\top} (p^{(t)} - y)$$



Le pas d'apprentissage

- **Trop petit** : la convergence est correcte mais lente. Le budget `max_iter` s'épuise avant l'optimum, et le modèle est sous-entraîné sans que rien ne le signale.
- **Adapté** : décroissance régulière de la fonction de coût.
- **Trop grand** : la suite oscille ou diverge. La condition de stabilité fait intervenir la plus grande valeur propre de la hessienne, $\eta < 2/\lambda_{\max}$.

Ce que fait scikit-learn

`LogisticRegression` n'utilise pas une descente de gradient simple mais un quasi-Newton (L-BFGS) qui adapte le pas automatiquement. L'utilisateur ne règle que `max_iter` et la tolérance. C'est pourquoi l'avertissement de non-convergence doit être lu, jamais ignoré.

On ajoute une pénalité à la fonction de coût :

$$\mathcal{L}_{\text{pen}}(\beta) = \mathcal{L}(\beta) + \frac{1}{C} \Omega(\beta), \quad \Omega_{L_2}(\beta) = \frac{1}{2} \|\beta\|_2^2, \quad \Omega_{L_1}(\beta) = \|\beta\|_1$$

L_2 (ridge)

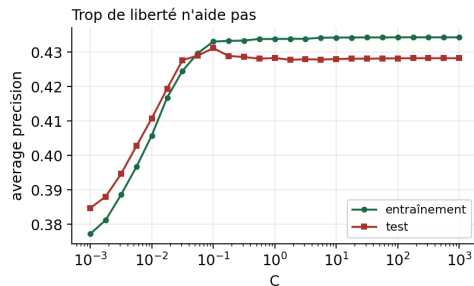
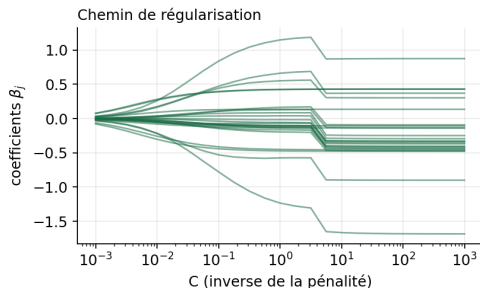
rétrécit tous les coefficients, stabilise en présence de colinéarité, ne sélectionne pas

L_1 (lasso)

annule exactement certains coefficients, donc sélectionne des variables

C est l'inverse de l'intensité : C petit signifie forte pénalité. Dans scikit-learn, L_2 est appliquée **par défaut** avec $C = 1$. Un modèle « sans régularisation » demande C très grand.

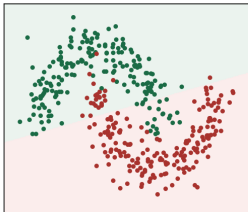
L'effet, sur nos données



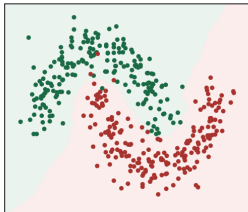
Les coefficients se rétractent vers zéro quand C diminue. Le score de test ne s'améliore pas au-delà d'un certain point.

Quatre familles d'algorithmes

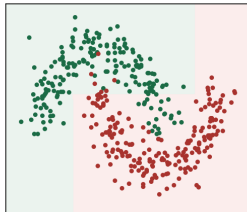
Régression logistique



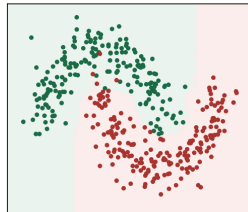
k plus proches voisins (k=15)



Arbre de décision (prof. 4)



Forêt aléatoire (300 arbres)



Logistique

k plus proches voisins

Arbre de décision

Forêt aléatoire

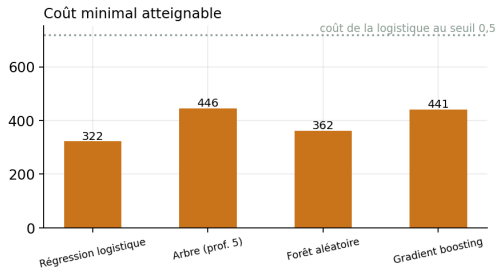
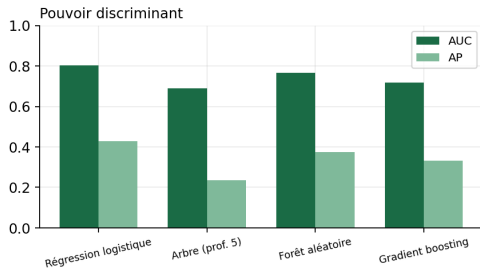
frontière linéaire, coefficients interprétables, probabilités calibrées

aucune phase d'apprentissage, coût à la prédiction, sensible à l'échelle

découpes parallèles aux axes, lisible, instable

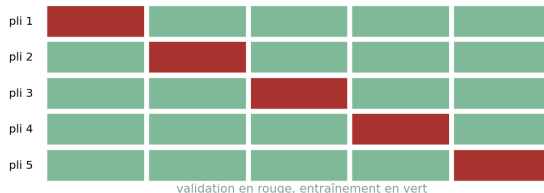
moyenne d'arbres décorrélés, variance réduite, moins lisible

Sur notre problème, que donnent-ils ?



La régression logistique reste devant. Sur données tabulaires de taille modeste, le modèle simple n'est pas un choix par défaut, c'est souvent le bon choix.

Validation croisée à 5 plis



k -fold stratifié : conserve la proportion de positifs dans chaque pli. Le choix par défaut en classification déséquilibrée.

Par groupe : quand plusieurs lignes appartiennent au même exploitant, elles doivent rester du même côté.

Temporel : on n'entraîne jamais sur le futur.

Régler les hyperparamètres sans tricher

```
from sklearn.model_selection import GridSearchCV

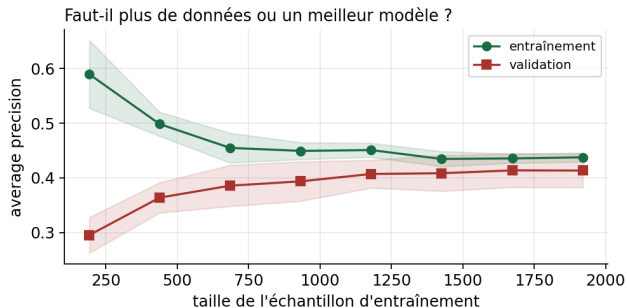
grille = {"clf__C": [0.01, 0.1, 1, 10, 100]}

rech = GridSearchCV(modele, grille, cv=5,
                    scoring="average_precision", n_jobs=-1)
rech.fit(X_train, y_train)
print(rech.best_params_, rech.best_score_)
```

La règle

La recherche se fait sur le **train**, par validation croisée interne. Le jeu de test n'est touché qu'une fois, à la toute fin. Chaque coup d'œil au test pour choisir quelque chose est une fuite qui rend le score final optimiste.

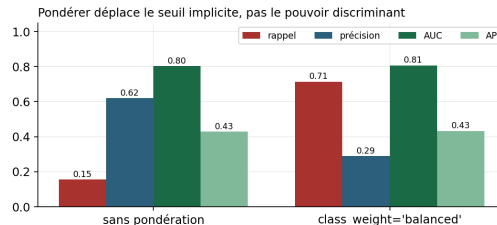
Plus de données, ou un meilleur modèle ?



Les deux courbes se rejoignent bas : le modèle est trop contraint. Ajouter des données n'aidera pas, il faut enrichir le modèle ou les variables.

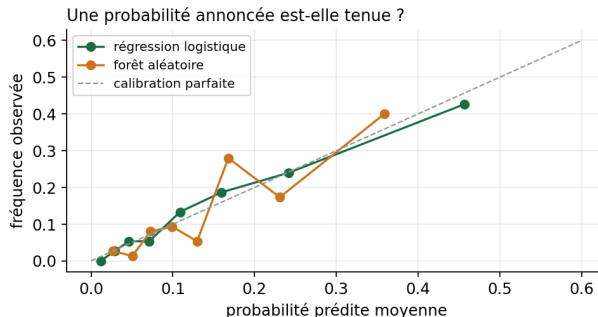
Un écart persistant : le modèle sur-apprend. Plus de données, ou davantage de régularisation.

1. **Déplacer le seuil** : gratuit, réversible, sans effet sur le modèle
2. **Pondérer les classes** : chaque positif compte $\frac{n}{2n_1}$ dans la fonction de coût
3. **Sur-échantillonner** : dupliquer ou interpoler les positifs (SMOTE)
4. **Sous-échantillonner** : jeter des négatifs, donc de l'information

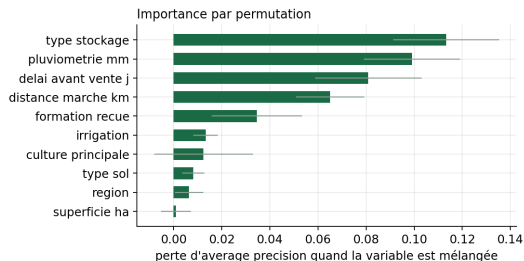


Pondérer déplace le seuil implicite. L'AUC bouge à peine : le pouvoir discriminant du modèle est inchangé.

Une probabilité annoncée est-elle tenue ?



Parmi les exploitations auxquelles le modèle attribue 0,30, en observe-t-on bien 30 % en perte élevée ? La régression logistique est naturellement bien calibrée, parce qu'elle maximise la vraisemblance. Les forêts et le boosting le sont beaucoup moins et demandent une recalibration (Platt, isotonique) si la probabilité elle-même est utilisée pour décider.



Importance par permutation

On mélange une colonne et on mesure la perte de performance. Applicable à n'importe quel modèle, y compris une boîte noire.

Coefficients

Propres à la logistique, lisibles en rapport de cotes, valables toutes choses égales par ailleurs.

Attention

Deux variables corrélées se partagent l'importance. Aucune des deux ne paraît importante alors que le couple l'est.

Corrélation et décision

Le modèle indique que l'absence de stockage *accompagne* un risque élevé. Il ne dit pas que construire un magasin *réduira* ce risque de tant.

Pour répondre à la seconde question, il faut une comparaison entre exploitations accompagnées et non accompagnées, tirées au sort ou rendues comparables. C'est un problème d'inférence causale, pas de prédiction.

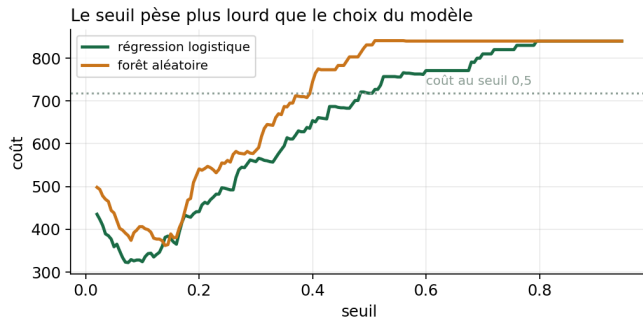
Un modèle prédictif sert à *cibler* une intervention. Il ne sert pas à *évaluer* son effet.

Étape	Question à laquelle il faut répondre
Exposer	Le modèle reçoit-il exactement les mêmes colonnes qu'à l'entraînement ?
Versionner	Quel jeu de données et quel code ont produit ce fichier <code>.joblib</code> ?
Surveiller	La distribution des entrées a-t-elle bougé depuis la mise en service ?
Réentraîner	Sur quel signal, à quelle fréquence, avec quelles nouvelles étiquettes ?
Auditer	Peut-on expliquer à un exploitant pourquoi il n'a pas été retenu ?

Ces questions sont le programme de mercredi et de jeudi. Elles supposent toutes acquis le protocole construit aujourd'hui.

Annexes

Extension : le seuil pèse plus que le modèle



Forêt aléatoire : AUC 0,766, coût minimal 362. Régression logistique : AUC 0,803, coût minimal 322.

Changer de modèle apporte moins que choisir correctement son seuil.

« Il faut rééquilibrer les classes »	C'est une option. Mais avant de modifier les données, modifiez le seuil : c'est gratuit, réversible, et ici cela divise le coût par deux.
« Pourquoi pas un réseau de neurones ? »	Sur 2 400 lignes tabulaires, il ferait moins bien pour dix fois le coût. Et il aurait exactement le même problème de seuil.
« Quel est le bon seuil ? »	Il n'y en a pas. Il y a un seuil pour un coût donné. Donnez-moi votre coût, je vous donne votre seuil.
« Les deux élevés à la fois ? »	Voir la courbe précision-rappel : le compromis est structurel, pas un défaut d'entraînement.

Taux de positifs	14,1 %
Baseline	0,860
Modèle, accuracy	0,868
Modèle, AUC	0,803
Modèle avec fuite	0,943
Précision au seuil 0,5	0,619
Rappel au seuil 0,5	0,155

Coût au seuil 0,5	718
Seuil optimal	0,075
Coût au seuil optimal	322
Rappel au seuil optimal	0,917
Seuil pour 150 visites	0,195
Rappel correspondant	0,595
Validation croisée	$0,413 \pm 0,031$

Jeu de test : 600 exploitations, dont 84 à risque.

- **Régularisation** L_1 et L_2 , choix du solveur
- **Optimisation d'hyperparamètres** : GridSearchCV, recherche aléatoire
- **Modèles d'ensemble** : forêts, gradient boosting
- **Rééquilibrage** : pondération des classes, sur-échantillonnage
- **Calibration** des probabilités
- **Déploiement** : mercredi et jeudi de cette semaine

Chacun de ces sujets suppose acquis ce que nous venons de construire.